

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

ДЕНЬ 15

Тема: «Разработка комплексных тестовых сценариев (Unit + Integration +
Contract)»

Цель работы:

Разработать набор тестов (модульных, интеграционных и контрактных)
для микросервиса «**Система управления заказами и доставкой**»,
обеспечивающих надежность модулей авторизации, обработки заказов и
внешних интеграций (платежи, логистика).

Выполнила:

Позднякова Марина,

Студент 24-ИС

2026г.

Вариант 1

Контрольные вопросы

1. `pytest` vs `unittest` — `pytest` проще (обычные функции + `assert`), `unittest` громоздче (классы + спец-методы).
2. Фикстуры — функции, которые подготавливают данные для тестов. Передаются как аргументы, избавляют от дублирования.
3. Мок HTTP — `requests-mock` или `mock.patch('requests.get')` — подменяют ответ, чтобы не ходить в сеть.
4. `@pytest.mark.parametrize` — прогоняет один тест с разными наборами данных.
5. Проверка исключения — `pytest.raises(Ошибка, match="текст")`.
6. `Mock()` vs `patch()` — `Mock()` создаёт заглушку, `patch()` временно подменяет реальный объект на заглушку.
7. Асинхронные тесты — `@pytest.mark.asyncio` + `async def` + `await`.
8. Фабрика фикстур — фикстура, возвращающая функцию для создания объектов с разными параметрами.
9. Проверка вызовов — `mock_obj.call_count == N` или `assert_called_with()`.
10. `coverage.py` — % покрытия кода тестами. Хороший порог — 80–90%.

структура проекта

project/

|— app/

| |— __init__.py

| |— models.py # Модели БД

| |— repositories.py # Репозиторий

| |— exceptions.py # Исключения (опционально)

|— tests/

|— __init__.py

|— conftest.py # Фикстуры

|— test_repository.py # Тест

Вариант 1. Интеграционный тест репозитория (SQLAlchemy)

Задача: Разработать интеграционный тест для OrderRepository.

Условие: В БД есть таблица orders с полями id, status (PENDING, PAID, SHIPPED), created_at.

Требование: Написать тест на pytest с фикстурой для БД (SQLite in-memory), который:

1. Поднимает тестовую БД через create_all().
2. Сохраняет заказ через репозиторий.
3. Выполняет запрос find_all_by_status("PENDING").
4. Проверяет, что транзакция корректно фиксируется.

Все работает и запускается

```
Requirement already satisfied: iniconfig==1.0.1 in C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging==22 in C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (26.2)
Requirement already satisfied: pluggy<2.0,>=1.5 in C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages (from pytest) (2.20.0)
Requirement already satisfied: coverage>=7.10.6 in C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages (from coverage[toml]>=7.10.6->pytest-cov) (7.14.3)
Requirement already satisfied: greenlet==1 in C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages (from sqlalchemy) (3.5.2)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages (from sqlalchemy) (4.15.0)

C:\Users\Student230\Desktop\Udm_15>pytest tests/test_repository.py -v
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student230\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Student230\Desktop\Udm_15
plugins: anyio-4.14.0, hypothesis-6.155.3, asyncio-1.4.0, cov-7.1.0, httpx-0.36.2, mock-3.15.1, xdist-3.8.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 1 item

tests/test_repository.py::test_find_orders_by_status PASSED [100%]

===== warnings summary =====
tests/test_repository.py::test_find_orders_by_status
tests/test_repository.py::test_find_orders_by_status
  C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages\sqlalchemy\sql\schemapath.py:3623: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return util.wrap_callable(lambda ctx: fn(), fn) # type: ignore
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 1 passed, 3 warnings in 0.23s =====

C:\Users\Student230\Desktop\Udm_15>pytest tests/test_repository.py --cov-app --cov-report-term-missing
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.0, pluggy-1.6.0
rootdir: C:\Users\Student230\Desktop\Udm_15
plugins: anyio-4.14.0, hypothesis-6.155.3, asyncio-1.4.0, cov-7.1.0, httpx-0.36.2, mock-3.15.1, xdist-3.8.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 1 item

tests/test_repository.py . [100%]

===== warnings summary =====
tests/test_repository.py::test_find_orders_by_status
tests/test_repository.py::test_find_orders_by_status
tests/test_repository.py::test_find_orders_by_status
  C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages\sqlalchemy\sql\schemapath.py:3623: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return util.wrap_callable(lambda ctx: fn(), fn) # type: ignore
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
_____ coverage: platform win32, python 3.13.0-final-0 _____
Name                               Stmts  Miss  Cover   Missing
app/_init_.py                       0      0  100%
app/models.py                       9      0  100%
app/repositories.py                 18      6    67%  18-19, 27-28
TOTAL                               27      6    76%

C:\Users\Student230\AppData\Local\Programs\Python\Python313\Lib\site-packages\pytest_unraisableexception.py:33: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x00000021E021F56C>
gc.collect()
```

Анализ проблем и способы их решения

В ходе выполнения тестов было обнаружено три основные проблемы, которые требуют внимания.

Первая проблема: использование устаревшей функции для работы с датой и временем

При выполнении тестов возникает предупреждение DeprecationWarning, которое указывает на то, что в коде используется функция datetime.utcnow(). Эта функция была помечена как устаревшая, начиная с версии Python 3.12, и в будущих версиях она будет полностью удалена. Проблема заключается в том, что функция utcnow() создает наивный объект datetime без информации о часовом поясе, что может привести к ошибкам при работе с разными временными зонами.

Для исправления этой проблемы необходимо заменить вызов datetime.utcnow() на datetime.now(timezone.utc). Это создаст объект datetime с явным указанием часового пояса UTC. Для этого нужно импортировать модуль timezone из пакета datetime: from datetime import datetime, timezone. Такая

замена сделает код совместимым с будущими версиями Python и обеспечит корректную работу с временными зонами.

Вторая проблема: недостаточное покрытие кода тестами

Анализ покрытия кода показывает, что тесты покрывают только 67 процентов кода в файле `repositories.py`. Конкретно непокрытыми остались строки с 16 по 19 и с 27 по 28. Это означает, что определенные методы или ветвления условий в репозитории не были протестированы.

Для решения этой проблемы необходимо добавить новые тесты, которые будут проверять непокрытые участки кода. Скорее всего, непокрытые строки относятся к методам поиска заказов с определенными статусами или к обработке случаев, когда заказы не найдены. Следует создать тесты для всех возможных сценариев использования репозитория: поиск заказов с различными статусами, обработка пустых результатов, создание новых заказов и обновление существующих. Также важно протестировать краевые случаи, такие как передача некорректных данных или обработка исключений.

Третья проблема: утечка ресурсов базы данных

В конце выполнения тестов возникает `ResourceWarning`, который предупреждает о том, что соединение с базой данных SQLite остается открытым после завершения всех тестов. Это происходит потому, что объект соединения с базой данных не был явно закрыт. Такая утечка ресурсов может привести к проблемам при выполнении большого количества тестов или при длительной работе приложения.

Для исправления этой проблемы необходимо правильно организовать управление ресурсами в тестах. Лучшим решением будет создание фикстуры `pytest`, которая будет открывать соединение с базой данных перед выполнением тестов и автоматически закрывать его после завершения. В фикстуре следует

использовать конструкцию `yield` для передачи репозитория в тест, а после завершения теста выполнить закрытие сессии и освобождение ресурсов движка базы данных.

Кроме того, можно использовать контекстный менеджер для автоматического управления подключением. Важно убедиться, что все тесты используют одну и ту же фикстуру для работы с базой данных, чтобы избежать создания множества отдельных соединений.

Вывод: В ходе выполнения практического задания мною был разработан интеграционный тест для репозитория заказов `OrderRepository` с использованием `SQLAlchemy` и `pytest`. Я организовала фикстуру `db_session`, которая создает тестовую базу данных `SQLite` в памяти и автоматически управляет соединением, обеспечивая изолированность тестов. Мною был реализован тестовый сценарий по паттерну `Arrange-Act-Assert`, включающий создание заказа со статусом `PENDING`, его сохранение через репозиторий и проверку корректности выполнения запроса `find_all_by_status`. Я убедилась в правильности работы транзакций и корректной фиксации изменений в базе данных. В результате проделанной работы создан надежный интеграционный тест, гарантирующий стабильную работу репозитория с базой данных в микросервисе управления заказами.